VCF 9.0 Undocumented Issues & Discoveries Reference

Version: 3.0 | **Date:** March 2026 | **Total Discoveries:** 35

> *Every issue in this document was discovered through independent lab investigation. None have official Broadcom KB articles, documentation, or known workarounds at the time of discovery. Each entry includes the exact problem, impact, and **complete copy-paste-ready resolution steps**.*

Table of Contents

1. Summary by Category
2. Database & Credential Operations (7 issues)
3. NSX in Nested/Resource-Constrained Environments (6 issues)
4. Certificate Management (5 issues)
5. VCF Operations 9.x Changes (6 issues)
6. Infrastructure & Platform (4 issues)
7. Crash Recovery & VCF Operations Suite-API (7 issues)
8. Quick Reference: All 35 Issues
9. Document Information

---

1. Summary by Category

Category	Issues	Severity Range	Key Impact
Database & Credential Operations	#1–7	Critical	All future credential ops blocked without DB repair
NSX in Nested Environments	#8–13	High	OOM crashes, boot storms, service instability
Certificate Management	#14–18	High	VDT failures, SDDC Manager trust broken

VCF Operations 9.x Changes	#19–24	Medium-High	Log/cert paths changed, adapters fail silently
Infrastructure & Platform	#25–28	Medium	vMotion failures, storage waste, missing tools
Crash Recovery & Suite-API	#29–35	High	Cannot manage VCF Ops without undocumented API formats

Discovery Timeline:

- Issues #1–28: Discovered during initial VCF 9.0.1 deployment (January–February 2026)
- Issues #29–35: Discovered during Windows Update crash recovery (March 2026)

---

2. Database & Credential Operations

Issue #1: SDDC Manager PostgreSQL Schema Unmapped

Problem: The SDDC Manager PostgreSQL database schema — table names, column names, and relationships — is completely undocumented by Broadcom.

Impact: Cannot troubleshoot credential failures, stuck tasks, or stale locks without knowing the schema.

Resolution — Map the schema yourself:

```
# Step 1: SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local
# Enter password when prompted: Success01!0909!!

# Step 2: Switch to root (needed for postgres access)
su -
# Enter root password: Success01!0909!!

# Step 3: Connect to PostgreSQL (MUST use -h 127.0.0.1 — see Issue #5)
sudo -u postgres psql -h 127.0.0.1 -d platform

# Step 4: List all tables in the platform database
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' ORDER BY table_name;

# Step 5: Get column details for any table
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
```

```
WHERE table_name = 'nsxt' ORDER BY ordinal_position;

# Step 6: Check a table's current content
SELECT * FROM nsxt;
SELECT * FROM lock;
SELECT id, status, resolved FROM task_metadata ORDER BY id DESC LIMIT 10;
```

Key tables discovered (complete reference):

Table	Purpose	Key Columns	Notes
`nsxt`	NSX cluster status	`id`, `state` (NOT `status`)	`state` must be `ACTIVE` for credential ops
`lock`	Resource-level locks	`resource_id`, `lock_type`, `created_at`	Stale locks block ALL operations
`task_metadata`	Task tracking	`id`, `resolved` (boolean), `status`	`resolved` must be `true` for completed tasks
`task_lock`	Task-level locks	`task_id`, `resource_id`	Links tasks to locked resources
`credential`	Managed credentials	`id`, `resource_type`, `account_type`	Credential inventory
`host`	ESXi hosts	`id`, `fqdn`, `status`	Commissioned host records

Issue #2: Credential Cascade Failure Mechanism

Problem: A failed credential rotation leaves NSX stuck in `ACTIVATING` state, stale locks accumulate, and unresolved tasks pile up. Each UI retry creates more locks, making it progressively worse.

Impact: All future credential operations are permanently blocked. The SDDC Manager UI shows errors on every credential operation.

How to identify this issue:

```
# Step 1: Get SDDC Manager API token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# Step 2: Check for stuck tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
```

```
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" | python3 -m
json.tool

# Step 3: Check for resource locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -m json.tool

# Step 4: Check NSX status (should be ACTIVE, not ACTIVATING)
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool
```

If you see: stuck IN_PROGRESS tasks, active resource locks, and/or NSX in ACTIVATING state — you have a credential cascade failure. **Follow Issue #4 for the complete 6-step fix.**

Issue #3: API Cannot Cancel Stuck Tasks

Problem: The SDDC Manager API returns `TA_TASK_CAN_NOT_BE_RETRIED` when attempting to retry stuck tasks. DELETE returns HTTP 500.

Impact: Database repair is the only fix path — the API provides no mechanism to resolve this.

How to confirm the API cannot help:

```
# Attempt to retry a stuck task (this will fail)
curl -sk -X PATCH "https://sddc-manager.lab.local/v1/tasks/<task-id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"status":"IN_PROGRESS"}'
# Response: {"errorCode":"TA_TASK_CAN_NOT_BE_RETRIED","message":"Task cannot be
retried"}

# Attempt to delete a stuck task (this will also fail)
curl -sk -X DELETE "https://sddc-manager.lab.local/v1/tasks/<task-id>" \
  -H "Authorization: Bearer $TOKEN"
# Response: HTTP 500
```

Resolution — Direct SQL fix:

```
# SSH to SDDC Manager and connect to PostgreSQL
ssh vcf@sddc-manager.lab.local
su -
```

```
sudo -u postgres psql -h 127.0.0.1 -d platform

# Find the stuck task ID
SELECT id, name, status, resolved FROM task_metadata
WHERE status = 'IN_PROGRESS' OR resolved = false
ORDER BY id DESC;

# Mark the specific stuck task as resolved
UPDATE task_metadata SET resolved = true WHERE id = '<task-id>';

# Or mark ALL unresolved tasks as resolved (nuclear option)
UPDATE task_metadata SET resolved = true WHERE resolved = false;

# Verify the fix
SELECT id, name, status, resolved FROM task_metadata
WHERE resolved = false;
-- Should return 0 rows

# Exit psql
\q
```

Issue #4: 6-Step PostgreSQL Repair Procedure

Problem: Fixing a credential cascade failure requires updating three tables in a specific sequence. Partial fixes still fail because all three tables participate in prevalidation. You MUST do all steps in order.

Impact: Without all 6 steps in order, the system remains broken.

Complete fix procedure (copy-paste every command):

```
# ================================================================
# SDDC MANAGER CREDENTIAL CASCADE FAILURE — COMPLETE FIX
# ================================================================
# Run each step in order. Do NOT skip steps.
# ================================================================

# STEP 1: SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local
# Password: Success01!0909!!

# Switch to root
su -
# Password: Success01!0909!!
```

```
# STEP 2: Enable trust authentication for PostgreSQL
# (Required because the PostgreSQL password is not discoverable)
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup

# Change "md5" to "trust" for local connections
sed -i 's/host    all.*127.0.0.1\/32.*md5/host    all             all
127.0.0.1\/32            trust/' \
  /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

# Restart PostgreSQL to pick up the change
systemctl restart postgresql

# STEP 3: Connect to the platform database
sudo -u postgres psql -h 127.0.0.1 -d platform

# STEP 4: Fix NSX cluster status (change ACTIVATING → ACTIVE)
SELECT id, state FROM nsxt;
-- If state shows 'ACTIVATING', run:
UPDATE nsxt SET state = 'ACTIVE' WHERE state = 'ACTIVATING';

# STEP 5: Clear ALL resource locks
SELECT * FROM lock;
-- Note how many rows exist (these are all stale)
DELETE FROM lock;
-- Should return: DELETE <count>

# STEP 6: Mark ALL unresolved tasks as resolved
SELECT id, name, status, resolved FROM task_metadata WHERE resolved = false;
-- Note the stuck tasks
UPDATE task_metadata SET resolved = true WHERE resolved = false;

# STEP 7: Clear task-level locks
SELECT * FROM task_lock;
DELETE FROM task_lock;

# STEP 8: Exit PostgreSQL
\q

# STEP 9: Restore md5 authentication (IMPORTANT — do not leave trust enabled)
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

systemctl restart postgresql
```

```
# STEP 10: Restart SDDC Manager operations service
systemctl restart operationsmanager

# Wait 5 minutes for operationsmanager to fully start

# STEP 11: Verify the fix — get a fresh token and check status
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# Should show: no IN_PROGRESS tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
  | python3 -c "import sys,json;print(len(json.load(sys.stdin).get('elements',
[])),'stuck tasks')"

# Should show: no resource locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks \
  | python3 -c "import sys,json;print(len(json.load(sys.stdin).get('elements',
[])),'locks')"

# Should show: NSX status = ACTIVE
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters \
  | python3 -c "import sys,json;d=json.load(sys.stdin);print(d['elements'][0]
['status'])"
```

Issue #5: PostgreSQL Requires TCP Connection

Problem: SDDC Manager PostgreSQL doesn't accept Unix socket connections. Must use `-h 127.0.0.1`.

Impact: `psql` without `-h` flag silently fails or connects to the wrong instance.

Resolution:

```
# WRONG — silently fails or connects to system PostgreSQL
sudo -u postgres psql -d platform
# Error: "FATAL: Peer authentication failed" or connects to wrong DB

# CORRECT — always use -h 127.0.0.1
```

```
sudo -u postgres psql -h 127.0.0.1 -d platform

# Verify you're connected to the right database
SELECT current_database();
-- Should return: platform

SELECT count(*) FROM nsxt;
-- Should return a row count (1 or more)
```

Issue #6: Database Column Naming Inconsistencies

Problem: The `nsxt` table uses `state` (not `status`), and `task_metadata` uses a `resolved` boolean (not a status enum). If you guess the column names wrong, your fix queries will silently do nothing.

Impact: Wrong column names = wrong queries = no fix.

Resolution — Use the correct column names:

```
-- WRONG — these columns don't exist and will error
UPDATE nsxt SET status = 'ACTIVE';           -- ERROR: column "status" does not
exist
UPDATE task_metadata SET status = 'RESOLVED'; -- This changes the wrong column

-- CORRECT column names
UPDATE nsxt SET state = 'ACTIVE' WHERE state = 'ACTIVATING';
UPDATE task_metadata SET resolved = true WHERE resolved = false;
```

Complete column reference:

Table	Column	Type	Valid Values
`nsxt`	`state`	varchar	`ACTIVE`, `ACTIVATING`, `ERROR`
`task_metadata`	`resolved`	boolean	`true`, `false`
`task_metadata`	`status`	varchar	`SUCCESSFUL`, `FAILED`, `IN_PROGRESS`
`lock`	`resource_id`	varchar	UUID of locked resource

Issue #7: PostgreSQL Password Not Discoverable

Problem: There is no documented method to obtain the SDDC Manager PostgreSQL password. The password is not stored in any accessible config file.

Impact: Cannot connect to the database for troubleshooting without a workaround.

Resolution — Trust authentication workaround:

```
# Step 1: SSH to SDDC Manager as root
ssh vcf@sddc-manager.lab.local
su -

# Step 2: Backup the current pg_hba.conf
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup

# Step 3: View current auth settings
cat /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf
# Look for the line: host  all  all  127.0.0.1/32  md5

# Step 4: Change md5 to trust for local connections
sed -i 's/host    all.*127.0.0.1\/32.*md5/host    all             all
127.0.0.1\/32            trust/' \
  /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

# Step 5: Restart PostgreSQL
systemctl restart postgresql

# Step 6: Now you can connect without a password
sudo -u postgres psql -h 127.0.0.1 -d platform

# Step 7: Do your work...

# Step 8: CRITICAL — Restore md5 auth when done
\q
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf
systemctl restart postgresql

# Step 9: Verify auth is restored (this should now fail)
psql -h 127.0.0.1 -U postgres -d platform
# Expected: "FATAL: password authentication failed" — this confirms md5 is back
```

> ***WARNING:*** *Do NOT leave trust authentication enabled. Always restore the backup pg_hba.conf after completing your database work. Trust auth allows anyone with local shell access to connect to the database without credentials.*

---

3. NSX in Nested/Resource-Constrained Environments

Issue #8: NSX Requires 32GB RAM Minimum

Problem: Broadcom documentation states 16GB minimum RAM for NSX Manager. In practice: 16GB = OOM kills, 24GB = intermittent crashes, 32GB = stable.

Impact: Under-provisioned NSX cascades into all VCF operations — credential rotation fails, SDDC Manager reports NSX as "UNSTABLE", and VDT fails.

Resolution — Set correct VM resources:

```
# Option A: Via vCenter UI
# 1. Power off NSX Manager VM
# 2. Right-click > Edit Settings
# 3. Set Memory: 32768 MB (32 GB)
# 4. Set CPU: 6 vCPU
# 5. Power on

# Option B: Via vCenter REST API
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n
'administrator@vsphere.local:Success01!0909!!' | base64)" \
  | tr -d '"')

# Power off the NSX Manager VM first
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-58/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# Wait for power off to complete (check status)
curl -sk -H "vmware-api-session-id: $SESSION" \
  "https://vcenter.lab.local/api/vcenter/vm/vm-58" \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['power_state'])"

# Update memory (API requires power off first)
curl -sk -X PATCH "https://vcenter.lab.local/api/vcenter/vm/vm-58" \
  -H "vmware-api-session-id: $SESSION" \
  -H "Content-Type: application/json" \
  -d '{"memory":{"size_MiB":32768}}'

# Power on
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-58/power?
```

```
action=start" \
  -H "vmware-api-session-id: $SESSION"
```

Tested configurations:

RAM	vCPU	Result
16 GB	4	OOM kills within 30 minutes
24 GB	6	Intermittent crashes under load
30 GB	6	Stable with occasional high load
32 GB	**6**	**Stable — recommended**

Issue #9: Boot Storm Load >100 is Normal

Problem: After power-on, NSX Manager experiences load averages exceeding 100 on 6 cores for 30-60 minutes. The VIP remains offline until services stabilize.

Impact: Credential operations triggered during boot storms cause cascade failures (Issue #2).

Resolution — Monitor and wait:

```
# SSH to NSX Manager (may take several attempts during boot storm)
ssh admin@192.168.1.71
# Password: Success01!0909!!

# Check load average
get node-stats

# From an ESXi host, monitor NSX VM's CPU
esxtop
# Press 'c' for CPU view, look for the NSX VM process

# Check if VIP is responding (run from any host with network access)
curl -sk --connect-timeout 5 https://nsx-vip.lab.local/api/v1/cluster/status \
  -u admin:'Success01!0909!!' 2>&1 | head -5
# If "Connection refused" or timeout — VIP is still coming up. Wait.

# Check service status from NSX CLI
get service
# Services should all show "running" after 30-60 min
```

Timeline after power-on:

Time	Expected State
0-5 min	SSH not responsive, VIP offline
5-15 min	SSH responds, load >50, services starting
15-30 min	Load 10-50, some services running
30-60 min	Load <10, all services stable, VIP online

> ***Rule:*** *Do NOT run credential rotations, certificate operations, or SDDC Manager tasks until load average drops below 10 and VIP is responding.*

Issue #10: Adding More vCPU is Counterproductive

Problem: Increasing NSX Manager vCPU count beyond 6 causes worse performance due to VMware co-scheduling overhead. ESXi must schedule all vCPUs simultaneously, meaning more vCPUs = harder to schedule = more wait time.

Impact: The intuitive fix (more CPU) actually makes the problem worse.

Resolution:

```
# If NSX Manager has >6 vCPU, reduce it:
# 1. Power off the NSX Manager VM
# 2. In vCenter UI: Right-click > Edit Settings
# 3. Set CPU to 6 vCPU
# 4. Power on

# If performance is still poor with 6 vCPU + 32GB RAM:
# Increase RAM to 48GB instead of adding more CPU
# The bottleneck is memory pressure from Java/Corfu, not CPU
```

Issue #11: Services Take 10-15 Minutes to Stabilize

Problem: After NSX Manager restart, services take 10-15 minutes to fully stabilize. The API returns `error 101` during this period.

Impact: Premature API calls fail and can trigger unnecessary retries. SDDC Manager may interpret the errors as a real failure and start cascading.

Resolution — Wait and verify:

```
# After restarting NSX Manager, run this loop from any machine with access:
for i in $(seq 1 30); do
  STATUS=$(curl -sk --connect-timeout 5 \
    -u admin:'Success01!0909!!' \
    https://nsx-vip.lab.local/api/v1/cluster/status 2>/dev/null \
    | python3 -c "import
sys,json;d=json.load(sys.stdin);print(d.get('control_cluster_status',
{}).get('status','UNAVAILABLE'))" 2>/dev/null || echo "UNREACHABLE")
  echo "[$(date +%H:%M:%S)] Cluster status: $STATUS"
  if [ "$STATUS" = "STABLE" ]; then
    echo "NSX Manager is ready."
    break
  fi
  sleep 30
done
```

Issue #12: DNS/NTP Must Be Set via Admin CLI

Problem: DNS and NTP settings must be configured using the NSX admin CLI, not the UI. UI settings don't persist in some nested configurations.

Impact: DNS resolution fails, causing certificate validation errors, SDDC Manager communication failures, and VDT failures.

Resolution — Set via CLI:

```
# SSH to NSX Manager
ssh admin@nsx-vip.lab.local
# Password: Success01!0909!!

# Set DNS server(s)
set name-servers 192.168.1.230

# Set NTP server(s)
set ntp-servers 192.168.1.230

# Verify DNS
nslookup vcenter.lab.local
nslookup sddc-manager.lab.local

# Verify NTP
```

```
get ntp-servers
get ntp-status
```

Issue #13: TEP on vmk0 (New in NSX 9.0)

Problem: NSX 9.0 supports "Use VMkernel Adapter" which reuses vmk0 as a Tunnel Endpoint (TEP), eliminating the need for a dedicated TEP VLAN. This is new in NSX 9.0 and not clearly documented.

Impact: Simplifies nested environments — no dedicated TEP VLAN required.

Resolution — Configure during Transport Node Profile creation:

```
In NSX Manager UI:
1. Navigate to: System > Fabric > Profiles > Transport Node Profiles
2. Click "Add Profile"
3. Under "Host Switch" configuration:
   - Type: VDS (Virtual Distributed Switch)
   - Mode: Standard
4. Under "IP Assignment":
   - Select: "Use VMkernel Adapter"
   - Select: vmk0
5. This reuses the management VMkernel as the TEP interface
6. No additional VLAN or IP pool configuration needed
```

> ***Note:*** *This is only recommended for nested lab environments or environments where a dedicated TEP VLAN is not available. Production environments should use a dedicated TEP VLAN for performance isolation.*

---

4. Certificate Management

Issue #14: NSX Cert SAN Must Include SDDC Manager's Registered FQDN

Problem: The NSX certificate Subject Alternative Names (SAN) must include the FQDN that SDDC Manager registered the NSX cluster with (e.g., `nsx-manager.lab.local`), NOT just the VIP and node FQDNs.

Impact: VDT reports SAN check failure. SDDC Manager loses trust in NSX. Certificate replacement appears successful but breaks SDDC Manager integration.

Resolution — Complete NSX certificate replacement procedure:

```
# ================================================================
# NSX CERTIFICATE REPLACEMENT — COMPLETE PROCEDURE
# ================================================================
# Run from NSX Manager SSH session (as root)
# ================================================================

# STEP 1: SSH to NSX Manager as root
ssh root@192.168.1.71
# Password: Success01!0909!!

# STEP 2: Create OpenSSL configuration file with ALL required SANs
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF

# CRITICAL: DNS.3 = nsx-manager.lab.local is REQUIRED because that's
# the FQDN SDDC Manager registered. Without it, VDT fails SAN check.

# STEP 3: Generate the certificate and private key
```

```
openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256

# STEP 4: Verify the SAN entries are correct
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"
# Expected output should show ALL 5 SANs:
#   DNS:nsx-vip.lab.local, DNS:nsx-node1.lab.local,
#   DNS:nsx-manager.lab.local, IP Address:192.168.1.70, IP Address:192.168.1.71

# STEP 5: Create JSON import payload using Python (avoids shell PEM escaping)
python3 -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json

# STEP 6: Import certificate into NSX
curl -k -u admin:'Success01!0909!!' \
  -X POST "https://192.168.1.71/api/v1/trust-management/certificates?
action=import" \
  -H "Content-Type: application/json" \
  -d @/tmp/nsx-import.json
# SAVE the certificate ID from the response, e.g.: 701d1416-5054-4038-8749-
4ac495980ebd

# STEP 7: Get the NSX node UUID
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster \
  | python3 -c "import sys,json;d=json.load(sys.stdin);print('Node
UUID:',d['nodes'][0]['node_uuid'])"
# SAVE the node UUID, e.g.: 95493642-ef4a-cb8e-ed7c-5bc20033f2c2

# STEP 8: Apply certificate to the NSX Manager node
# Replace <CERT-ID> and <NODE-UUID> with actual values from steps 6 and 7
curl -k -u admin:'Success01!0909!!' \
  -X POST "https://192.168.1.71/api/v1/trust-management/certificates/<CERT-ID>?
action=apply_certificate&service_type=API&node_id=<NODE-UUID>"
# NSX Manager will restart — wait 2-3 minutes

# STEP 9: Apply certificate to the cluster VIP
curl -k -u admin:'Success01!0909!!' \
  -X POST "https://192.168.1.71/api/v1/trust-management/certificates/<CERT-ID>?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

```
# STEP 10: Verify the new certificate is active
openssl s_client -connect 192.168.1.71:443 -showcerts </dev/null 2>/dev/null \
  | openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
openssl s_client -connect 192.168.1.70:443 -showcerts </dev/null 2>/dev/null \
  | openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

Then import into SDDC Manager trust stores (see Issue #15).

Issue #15: Two Separate Trust Stores Must Be Updated

Problem: VCF has two separate trust stores that both need CA cert imports: the VCF common services trust store and the Java cacerts keystore. KB 316056 only documents one of them.

Impact: Missing either import causes VDT failures and inter-component trust issues.

Resolution — Import into BOTH trust stores on SDDC Manager:

```
# SSH to SDDC Manager as root
ssh vcf@sddc-manager.lab.local
su -

# STEP 1: Pull the active NSX certificate
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null \
  | openssl x509 -outform PEM > /tmp/nsx-root.crt

# Verify you got a valid certificate
openssl x509 -in /tmp/nsx-root.crt -noout -subject -issuer -dates

# STEP 2: Import into VCF common services trust store
# First, get the keystore password
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
echo "Keystore password: $KEY"

# Import the certificate
keytool -importcert -alias nsx-selfsigned \
  -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt

# Verify it was imported
keytool -list -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" | grep nsx-selfsigned
```

```
# STEP 3: Import into Java cacerts trust store
keytool -importcert -alias nsx-selfsigned \
  -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Verify it was imported
keytool -list -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit | grep nsx-selfsigned

# STEP 4: Restart SDDC Manager services to pick up the new trust
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh

# Wait approximately 5 minutes for all services to restart

# STEP 5: Verify trust by re-running VDT
# (See Issue #28 for VDT installation if not already installed)
```

Issue #16: Fleet Management Cert Generator Produces Wrong SANs

Problem: The Fleet Management deployment wizard's "Generate self-signed certificate" option produces a certificate whose SAN entries do not match the node FQDN/IP, causing a precheck error: "*Certificate validation for component — The hosts in the certificate doesn't match with the provided/product hosts.*"

Impact: Fleet Management deployment wizard precheck fails.

Resolution — Generate a correct certificate manually with OpenSSL:

```
# Run on SDDC Manager (SSH as root) or any Linux host with openssl

# STEP 1: Create OpenSSL config for Fleet Management
# Replace fleet.lab.local / 192.168.1.78 with your Fleet FQDN/IP
cat > /tmp/fleet-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
```

```
ST = California
L = Lab
O = Lab
OU = VCF
CN = fleet.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = fleet.lab.local
DNS.2 = fleet
IP.1 = 192.168.1.78
EOF

# STEP 2: Generate certificate and key
openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/fleet.key -out /tmp/fleet.crt \
  -config /tmp/fleet-cert.cnf

# STEP 3: Verify SANs are correct
openssl x509 -in /tmp/fleet.crt -noout -text | grep -A5 "Subject Alternative Name"
# Expected: DNS:fleet.lab.local, DNS:fleet, IP Address:192.168.1.78

# STEP 4: Display cert and key for copy-paste into the wizard
echo "=== CERTIFICATE ==="
cat /tmp/fleet.crt
echo ""
echo "=== PRIVATE KEY ==="
cat /tmp/fleet.key

# STEP 5: In the Fleet Management deployment wizard:
# 1. At the "Certificate" step, select "Import"
# 2. Paste the certificate content (fleet.crt) into the Certificate field
# 3. Paste the key content (fleet.key) into the Private Key field
# 4. Click "Validate"
# 5. Continue to Component Configuration
# 6. Run Precheck — should now pass
```

Issue #17: VCF Ops for Logs Cert Generator — Same SAN Mismatch

Problem: Identical pattern to Issue #16 — the VCF Operations for Logs certificate generator produces wrong SANs.

Impact: Logs deployment wizard precheck fails with the same "hosts in the certificate doesn't match" error.

Resolution — Generate a correct certificate manually with OpenSSL:

```
# Run on SDDC Manager (SSH as root) or any Linux host with openssl

# STEP 1: Create OpenSSL config for VCF Ops for Logs
# Replace logs.lab.local / 192.168.1.242 with your Logs FQDN/IP
cat > /tmp/vrli-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
EOF

# STEP 2: Generate certificate and key
openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/vrli.key -out /tmp/vrli.crt \
  -config /tmp/vrli-cert.cnf

# STEP 3: Verify SANs are correct
```

```
openssl x509 -in /tmp/vrli.crt -noout -text | grep -A5 "Subject Alternative Name"
# Expected: DNS:logs.lab.local, DNS:logs, IP Address:192.168.1.242

# STEP 4: Display cert and key for copy-paste into the wizard
echo "=== CERTIFICATE ==="
cat /tmp/vrli.crt
echo ""
echo "=== PRIVATE KEY ==="
cat /tmp/vrli.key

# STEP 5: In the Logs deployment wizard:
# 1. At the "Certificate" step, select "Import"
# 2. Paste the certificate (vrli.crt) and key (vrli.key)
# 3. Click "Validate"
# 4. Run Precheck — should now pass

# STEP 6: After deployment, verify the cert on the deployed appliance
openssl s_client -connect logs.lab.local:443 -servername logs.lab.local \
  </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

Issue #18: Shell Can't Handle PEM Escaping for NSX API

Problem: When importing certificates via the NSX REST API, the JSON payload requires PEM certificate content with proper newline escaping. Bash/curl with inline PEM content breaks because PEM files contain newlines that JSON requires escaped as `\n`.

Impact: Cannot import certificates via a simple curl command.

Resolution — Use Python to construct and send the JSON payload:

```
# OPTION A: Python script (recommended)
python3 << 'PYEOF'
import json, requests, urllib3
urllib3.disable_warnings()

# Read cert and key files
with open('/tmp/nsx.crt') as f:
    cert = f.read()
with open('/tmp/nsx.key') as f:
    key = f.read()

# Construct JSON payload — Python handles the escaping automatically
payload = {"pem_encoded": cert, "private_key": key}
```

```
# Import certificate
resp = requests.post(
    "https://192.168.1.71/api/v1/trust-management/certificates?action=import",
    auth=("admin", "Success01!0909!!"),
    json=payload,
    verify=False
)
print(f"Status: {resp.status_code}")
print(f"Response: {resp.text}")
# Save the certificate ID from the response
PYEOF

# OPTION B: Create JSON file first, then use curl
python3 -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json

# Then use curl with the JSON file
curl -k -u admin:'Success01!0909!!' \
  -X POST "https://192.168.1.71/api/v1/trust-management/certificates?
action=import" \
  -H "Content-Type: application/json" \
  -d @/tmp/nsx-import.json
```

Why this matters: *If you try to embed PEM content directly in a curl* `-d` *argument, the newlines in the PEM file break the JSON structure. Python's* `json.dumps()` *properly escapes* `\n` *characters automatically.*

---

5. VCF Operations 9.x Changes

Issue #19: Adapter Log Paths Changed

Problem: VCF Operations 9.x changed adapter log paths. The legacy path documented in older guides doesn't exist.

Impact: Cannot find logs for adapter troubleshooting.

Resolution — Use the correct paths:

```
# SSH to VCF Operations node
ssh root@192.168.1.77
# Password: Success01!0909!!

# CORRECT path for adapter logs in VCF Ops 9.x
ls /storage/log/vcops/log/adapters/

# View a specific adapter's log
ls /storage/log/vcops/log/adapters/
# Pick the adapter directory, e.g., VMware_NSXTAdapter/
tail -100 /storage/log/vcops/log/adapters/VMware_NSXTAdapter/*.log

# WRONG legacy path (does not exist in 9.x)
ls /usr/lib/vmware-vcops/user/plugins/inbound/*/logs/
# Error: No such file or directory

# Other useful log locations
tail -100 /usr/lib/vmware-casa/casa-webapp/logs/casa.log
tail -100 /storage/log/vcops/log/vcops-admin.log
```

Issue #20: JRE Path Changed

Problem: The JRE path changed to `/usr/java/jre-vmware-17/`. The legacy `jre-vmware` symlink doesn't exist.

Impact: Cannot import certificates into the correct truststore; keytool commands fail.

Resolution — Use the correct JRE path:

```
# SSH to VCF Operations node
ssh root@192.168.1.77

# CORRECT JRE path in VCF Ops 9.x
ls /usr/java/jre-vmware-17/

# CORRECT cacerts path
ls -la /usr/java/jre-vmware-17/lib/security/cacerts

# Import a CA certificate into the correct truststore
keytool -import -trustcacerts -alias my-ca \
  -file /tmp/ca-cert.pem \
  -keystore /usr/java/jre-vmware-17/lib/security/cacerts \
```

```
  -storepass changeit -noprompt

# List certificates in the truststore
keytool -list -keystore /usr/java/jre-vmware-17/lib/security/cacerts \
  -storepass changeit | grep my-ca

# WRONG legacy path (does not exist)
ls /usr/java/jre-vmware/
# Error: No such file or directory
```

Issue #21: Two Separate NSX Adapters

Problem: VCF Operations has two separate NSX adapters — the VCF section auto-creates one using the VIP, while the "Aria Admin" section uses the individual node FQDN.

Impact: Both need separate credentials configured. The Aria Admin adapter may continue working when the VIP is down.

Resolution — Configure both adapters:

```
# Get Suite-API token
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# List all adapters to see both NSX adapters
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for a in d.get('adapterInstancesInfoDto',[]):
    kind=a.get('resourceKey',{}).get('adapterKindKey','')
    if 'NSX' in kind.upper():
        name=a.get('resourceKey',{}).get('name','?')
        aid=a.get('id','?')
        print(f'ID: {aid} | Name: {name} | Kind: {kind}')
"

# You will see two NSX-related adapters:
# 1. VCF adapter's auto-discovered NSX (uses VIP: nsx-vip.lab.local)
```

```
# 2. Standalone NSXTAdapter (may use node FQDN: nsx-manager.lab.local)
# Both need valid credentials to function properly
```

Issue #22: System Managed Credential ROTATE Doesn't Work for NSX

Problem: The system-managed credential rotation for the NSX adapter silently fails — the credential is not actually rotated, but no error is shown.

Impact: NSX monitoring stops when the password changes but the adapter still has the old password.

Resolution — Set credentials manually:

```
In VCF Operations Admin UI (https://192.168.1.77/):
1. Navigate to: Administration > Solutions > Adapters
2. Find the NSX adapter (NSXTAdapter)
3. Click the adapter name to edit
4. Under Credential:
   - UNCHECK "System Managed"
   - Enter the username: admin
   - Enter the current password manually
5. Click Save
6. Click "Test Connection" to verify
7. If the connection test passes, the adapter will resume data collection
```

Issue #23: SSH Enable via Admin UI Only

Problem: SSH access to VCF Operations can only be enabled through the Admin UI at `https://<vcf-ops>:443/admin/`. Console and systemctl approaches don't work.

Impact: Cannot SSH for troubleshooting without Admin UI access first.

Resolution:

```
1. Open a browser and navigate to: https://192.168.1.77/admin/
2. Log in with:
   - Username: admin
   - Password: Success01!0909!!
3. Navigate to: Administration > Access > SSH
4. Toggle SSH to: Enabled
5. Click Save

# Now you can SSH:
```

```
ssh root@192.168.1.77
# Password: Success01!0909!!
```

> ***If Admin UI is not accessible:*** *You must use the VM console (vCenter > VM > Launch Console) to access the appliance. From the console, the* `admin` *user can access the Admin UI settings.*

Issue #24: Health Adapter Silently Fails on Stale Credential

Problem: The VMWARE_INFRA_HEALTH adapter silently fails when an SDDC Manager credential becomes stale (e.g., after password rotation). The UI stop/start does not fix it.

Impact: Health monitoring stops. No alerts, no health data collection.

Resolution — Full appliance reboot required:

```
# Step 1: Verify the health adapter is actually failing
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for a in d.get('adapterInstancesInfoDto',[]):
    kind=a.get('resourceKey',{}).get('adapterKindKey','')
    if 'HEALTH' in kind:
        name=a.get('resourceKey',{}).get('name','?')
        status=a.get('adapter-status',{}).get('adapterStatus','?')
        print(f'{name}: {status}')
"

# Step 2: Try stop/start first (usually doesn't work but worth trying)
# Find the adapter ID from the output above, then:
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/stop"
\
  -H "Authorization: vRealizeOpsToken $TOKEN"
sleep 10
curl -sk -X PUT \
```

```
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start"
\
  -H "Authorization: vRealizeOpsToken $TOKEN"

# Step 3: If stop/start doesn't fix it, reboot the appliance
ssh root@192.168.1.77
reboot

# Wait 10-15 minutes for the appliance to fully restart
# Then verify the health adapter is collecting data again
```

---

6. Infrastructure & Platform

Issue #25: vCenter Can't Thin-Provision to vSAN

Problem: The vCenter storage migration wizard keeps thick provisioning even when thin is selected. In this lab, SDDC Manager had 914GB allocated but only 108GB of actual data.

Impact: Massive storage waste on vSAN. A single VM can consume 10x more space than it actually needs.

Resolution — Use vmkfstools to clone each disk as thin:

```
# ================================================================
# THICK-TO-THIN MIGRATION — COMPLETE PROCEDURE
# ================================================================
# Example: SDDC Manager (6 disks, 914GB thick → ~108GB thin)
# ================================================================

# STEP 1: Power off the VM in vCenter (UI or API)
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n
'administrator@vsphere.local:Success01!0909!!' | base64)" \
  | tr -d '"')
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-68/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# STEP 2: SSH to the ESXi host where the VM is registered
ssh root@192.168.1.201
# Password: Success01!0909!!
```

```
# STEP 3: Create destination directory on vSAN
mkdir -p /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/

# STEP 4: Clone each disk as thin provisioned
# Syntax: vmkfstools -i <source.vmdk> <dest.vmdk> -d thin
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_1.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_1.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_2.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_2.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_3.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_3.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_4.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_4.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_5.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_5.vmdk -d thin

# NOTE: If a clone fails partway through (e.g., host disconnect), delete the
# partial copy before retrying:
# vmkfstools -U /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-
manager_3.vmdk
# Then retry the clone command.

# STEP 5: Copy configuration files (VMX, NVRAM, VMSD)
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmx \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.nvram \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmsd \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/

# STEP 6: Verify thin provisioned disks
du -sh /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
# Should show ~108GB instead of 914GB

# STEP 7: In vCenter UI:
# - Right-click original VM > "Remove from Inventory" (NOT Delete from Disk)
# - Navigate to Datastores > vSAN > Browse > sddc-manager/
```

```
# - Right-click sddc-manager.vmx > "Register VM"
# - Power on and verify
```

Issue #26: `vhv.enable` Ghost Setting

Problem: The `vhv.enable` setting persists in a VM's runtime DICT (vmware.log) even when it is not present in the VMX file. This causes vMotion to fail with: "*The virtual machine cannot be restored because the snapshot was taken with VHV enabled.*"

Impact: vMotion fails with a confusing error. The vCenter UI shows "Expose hardware assisted virtualization" unchecked, and the VMX file has no `vhv.enable` entry — yet the setting is active.

Resolution — Explicitly set FALSE in the VMX file:

```
# Step 1: Power off the VM

# Step 2: SSH to the ESXi host where the VM resides
ssh root@192.168.1.201

# Step 3: Check if vhv.enable exists in the VMX file
grep -i vhv /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmx
# If no output, the setting is NOT in the file (but may be in runtime)

# Step 4: Add explicit FALSE — even if the line doesn't exist
echo 'vhv.enable = "FALSE"' >> /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-
manager/sddc-manager.vmx

# Step 5: Verify it was added
grep -i vhv /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmx
# Should show: vhv.enable = "FALSE"

# Step 6: Power on the VM and retry vMotion
```

Key lesson: *The ABSENCE of* `vhv.enable` *in the VMX file does NOT mean it is disabled. The setting can persist from a previous deployment environment. You must always add an explicit* `vhv.enable = "FALSE"` *to fix vMotion failures.*

Issue #27: Hot vMotion Fails in Nested Environments

Problem: Live vMotion (hot migration) fails due to memory convergence timeout. The hypervisor cannot converge the memory pages fast enough through the nested network stack.

Impact: Must use cold migration as a fallback. This means VM downtime during migration.

Resolution — Use cold migration:

```
# Step 1: Power off the VM
# Via vCenter UI: Right-click VM > Power > Shut Down Guest OS
# Or via API:
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n
'administrator@vsphere.local:Success01!0909!!' | base64)" \
  | tr -d '"')
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# Step 2: In vCenter UI:
# Right-click VM > Migrate
# Select "Change both compute resource and storage"
# Select destination host/datastore
# The migration will proceed as a cold migration (relocate powered-off VM)

# Step 3: Power on VM at the new location
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?
action=start" \
  -H "vmware-api-session-id: $SESSION"
```

Alternative: *If you must avoid downtime, try increasing the vMotion timeout in the advanced settings:* `Host > Configuration > Advanced Settings > Migrate.Enabled = 1` *and* `Migrate.PreCopyAbsoluteMaxRound = 200` *. This may help in some cases but is not guaranteed for nested environments.*

Issue #28: VDT Not Pre-Installed on SDDC Manager

Problem: The VMware Deployment Toolkit (VDT) is not pre-installed on SDDC Manager. Must download from Broadcom KB 344917 and upload manually.

Impact: Cannot run health checks or validation without manual setup.

Resolution — Download, upload, install, and run VDT:

```
# STEP 1: Download VDT from Broadcom
# Go to: https://knowledge.broadcom.com/external/article/344917
# Download the latest VDT zip file (e.g., vdt-2.2.7_02-05-2026.zip)

# STEP 2: Upload to SDDC Manager via SCP
# From your workstation:
scp vdt-2.2.7_02-05-2026.zip vcf@sddc-manager.lab.local:/tmp/
# Password: Success01!0909!!

# STEP 3: SSH to SDDC Manager and install
ssh vcf@sddc-manager.lab.local
su -

cd /tmp
unzip vdt-2.2.7_02-05-2026.zip -d /opt/vmware/vdt/

# STEP 4: Run VDT
cd /opt/vmware/vdt/
python3 vdt.py

# VDT will check:
# - DNS resolution (forward and reverse)
# - NTP synchronization
# - Certificate validity and SAN matching
# - Service health
# - Password status
# - Component connectivity

# STEP 5: Review results
# VDT creates a report at:
cat /var/log/vmware/vcf/vdt/vdt-*.txt
```

---

7. Crash Recovery & VCF Operations Suite-API

These issues were discovered during the Windows Update crash recovery in March 2026.

Issue #29: VCF Operations Suite-API Uses Non-Standard Auth Header

Problem: The VCF Operations Suite-API requires the auth header format `vRealizeOpsToken <token>` — NOT `Bearer` or `VMware` like every other VMware API.

Impact: All API calls fail with 401 Unauthorized if using the standard `Bearer` format. Error message does not explain the correct format.

Resolution — Complete working example:

```
# STEP 1: Get authentication token
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

echo "Token: $TOKEN"

# STEP 2: Use the token with the CORRECT header format
# WRONG — returns 401:
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users

# CORRECT — returns 200:
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users | python3 -m json.tool

# All subsequent API calls must use "vRealizeOpsToken" prefix:
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters | python3 -m json.tool

curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/deployment/node/status | python3 -m json.tool

curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/collectors | python3 -m json.tool
```

Issue #30: VCF Operations Permissions API Requires Single Object (Not Array)

Problem: The PUT body for user permissions at `/suite-api/api/auth/users/{id}/permissions` must be a single JSON object with `roleName`, NOT wrapped in an array, `permissions` key, or any other wrapper.

Impact: Every other format returns "Role with name: null cannot be found" — an unhelpful error that doesn't indicate the format is wrong.

Resolution — Complete working example:

```
# STEP 1: Get the user ID you want to modify
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for u in d.get('users',[]):
    print(f\"ID: {u['id']} | Username: {u['username']} | Roles:
{u.get('roleNames',[])}\")"

# STEP 2: List available roles
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/roles \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for r in d.get('roles',[]):
    print(f\"Role: {r['name']} | Privileges: {len(r.get('privilege-keys',[]))}\")"

# STEP 3: Assign the Administrator role to a user
# Replace <USER-ID> with the actual user ID from step 1

# CORRECT format — single JSON object, NOT an array:
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/auth/users/<USER-ID>/permissions" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{"roleName":"Administrator","allowAllObjects":true,"traversal-spec-
instances":[]}'

# WRONG formats that all return "Role with name: null":
# {"permissions": [{"roleName": "Administrator"}]}
# [{"roleName": "Administrator"}]
# {"roleName": "Administrator", "permissions": []}
# {"role": {"name": "Administrator"}}

# STEP 4: Verify the role was assigned
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  "https://192.168.1.77/suite-api/api/auth/users/<USER-ID>" \
  | python3 -c "import
sys,json;d=json.load(sys.stdin);print('Roles:',d.get('roleNames',[]))"
```

Issue #31: VCF Operations Super Admin Shows Empty Roles (By Design)

Problem: The built-in `admin` user always shows `roleNames: []` in the Suite-API. This looks like a bug but is by design.

Impact: Administrators waste time trying to "fix" the admin role assignment. Any attempt to modify the admin user fails.

Resolution — No action needed. Confirm it's working:

```
# Verify admin has implicit full access despite empty roles
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# These all work — proving admin has full access:
echo "Users (admin-only):"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users -o /dev/null -w "%{http_code}\n"

echo "Adapters:"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters -o /dev/null -w "%{http_code}\n"

echo "Cluster status:"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/deployment/node/status -o /dev/null -w "%
{http_code}\n"
# All should return: 200

# These will FAIL (by design):
# PUT to modify admin user → HTTP 500 "Cannot create or update super admin"
# DELETE admin user → HTTP 500 "system created and cannot be deleted"
# Neither of these is a bug — admin is a protected super admin account
```

Issue #32: SDDC Manager domainmanager Uses HTTP (Not HTTPS) on Port 7200

Problem: The `domainmanager` service on SDDC Manager listens on port 7200 using plain **HTTP** (not HTTPS). Using `curl -sk https://localhost:7200` fails with "wrong version number" — a confusing error that suggests a TLS problem.

Impact: You waste time troubleshooting TLS when the real issue is just wrong protocol.

Resolution:

```
# SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local

# WRONG — misleading "wrong version number" error:
curl -sk https://localhost:7200/health
# Error: curl: (35) error:1408F10B:SSL routines:ssl3_get_record:wrong version
number

# CORRECT — use HTTP:
curl -s http://localhost:7200/health
# Returns: {"status":"UP"}

# All internal service ports and their protocols:
# Port 7200 — domainmanager — HTTP  (not HTTPS!)
# Port 7300 — operationsmanager — HTTP
# Port 7400 — lcm — HTTP
# Port 443  — Nginx reverse proxy — HTTPS (this is what external clients use)

# Quick check for all services:
for port in 7200 7300 7400; do
  STATUS=$(curl -s --connect-timeout 3 http://localhost:$port/health 2>/dev/null)
  echo "Port $port: $STATUS"
done
```

Issue #33: NSX Adapter Credential Fields Must Use Uppercase Names

Problem: When creating NSX adapter credentials via Suite-API, the field names must be `USERNAME` and `PASSWORD` (all uppercase). Using `USER` or `user` fails with "USERNAME is mandatory".

Impact: No documentation specifies the exact field names. Trial and error is the only way to discover this.

Resolution — Complete credential and adapter creation:

```
# STEP 1: Create the credential with CORRECT field names
CRED_RESP=$(curl -sk -X POST "https://192.168.1.77/suite-api/api/credentials" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "nsx-vip.lab.local",
    "adapterKindKey": "NSXTAdapter",
```

```
    "credentialKindKey": "NSXTCREDENTIAL",
    "fields": [
      {"name": "USERNAME", "value": "admin"},
      {"name": "PASSWORD", "value": "Success01!0909!!"}
    ]
  }')

echo "$CRED_RESP" | python3 -m json.tool
CRED_ID=$(echo "$CRED_RESP" | python3 -c "import
sys,json;print(json.load(sys.stdin)['id'])")
echo "Credential ID: $CRED_ID"

# WRONG field names that will fail:
# {"name": "USER", "value": "admin"}        → "USERNAME is mandatory"
# {"name": "user", "value": "admin"}        → "USERNAME is mandatory"
# {"name": "username", "value": "admin"}    → "USERNAME is mandatory"
# {"name": "PASS", "value": "..."}          → "PASSWORD is mandatory"

# STEP 2: Create the NSX adapter using the credential
curl -sk -X POST "https://192.168.1.77/suite-api/api/adapters" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d "{
    \"name\": \"nsx-vip.lab.local\",
    \"description\": \"NSX Manager\",
    \"adapterKindKey\": \"NSXTAdapter\",
    \"resourceIdentifiers\": [
      {\"name\": \"NSXTHOST\", \"value\": \"nsx-vip.lab.local\"},
      {\"name\": \"AUTO_DISCOVERY\", \"value\": \"true\"},
      {\"name\": \"ENABLE_ALERTS_FROM_NSX\", \"value\": \"false\"},
      {\"name\": \"VCURL\", \"value\": \"vcenter.lab.local\"},
      {\"name\": \"VMEntityVCID\", \"value\": \"92109cf0-ad3b-4ffa-8972-
a77bb7fadacf\"},
      {\"name\": \"NSX_CLUSTER_ID\", \"value\": \"6c55d856-ab96-4190-8495-
3cc8cb23450c\"}
    ],
    \"credential\": {\"id\": \"$CRED_ID\"},
    \"collectorId\": 2
  }"
```

Issue #34: Gemfire Cache Takes 5-10 Minutes to Populate After Cluster Init

Problem: After VCF Operations cluster initialization, the Gemfire distributed cache takes 5-10 minutes to fully populate. Roles, users, adapters, and other data may not appear in API responses during this window.

Impact: Administrators conclude data is missing and take unnecessary corrective action (like trying to recreate roles or reinitialize the cluster).

Resolution — Wait and verify:

```
# After cluster initialization, run this monitoring loop:
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

for i in $(seq 1 20); do
  ROLES=$(curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
    https://192.168.1.77/suite-api/api/auth/roles 2>/dev/null \
    | python3 -c "import sys,json;d=json.load(sys.stdin);print(len(d.get('roles',
[])))" 2>/dev/null || echo "0")
  ADAPTERS=$(curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
    https://192.168.1.77/suite-api/api/adapters 2>/dev/null \
    | python3 -c "import
sys,json;d=json.load(sys.stdin);print(len(d.get('adapterInstancesInfoDto',[])))"
2>/dev/null || echo "0")
  echo "[$(date +%H:%M:%S)] Roles: $ROLES | Adapters: $ADAPTERS"
  if [ "$ROLES" -gt 0 ] && [ "$ADAPTERS" -gt 0 ] 2>/dev/null; then
    echo "Gemfire cache populated — system ready."
    break
  fi
  sleep 30
done
```

Expected timeline after cluster init:

Time	Roles Visible	Adapters Visible
0-2 min	0	0
2-5 min	0-3	0-5
5-10 min	All (e.g., 5)	All (e.g., 15+)

Issue #35: VCF Operations HSQLDB Reset Required After Unclean Shutdown

Problem: An unclean shutdown of VCF Operations leaves the HSQLDB and Gemfire cache in an inconsistent state, causing `INITIALIZATION_FAILED`. There is no automatic recovery mechanism.

Impact: VCF Operations is completely non-functional until manual HSQLDB reset.

Resolution — Complete HSQLDB reset procedure:

```
# ================================================================
# VCF OPERATIONS HSQLDB RESET — COMPLETE PROCEDURE
# ================================================================

# STEP 1: SSH to VCF Operations node as root
ssh root@192.168.1.77
# Password: Success01!0909!!

# STEP 2: Verify the problem — check cluster state
curl -sk https://localhost/casa/cluster/status
# Expected: "state": "INITIALIZATION_FAILED"

# STEP 3: Stop all VCF Operations services
systemctl stop vmware-casa
systemctl stop vmware-vcops-watchdog

# STEP 4: Backup the HSQLDB script file
cp /storage/db/casa/webapp/hsqldb/casa.db.script \
   /storage/db/casa/webapp/hsqldb/casa.db.script.bak

# STEP 5: Edit the HSQLDB — change initialization state
# Find the line containing "initialization_state":"FAILED"
grep -n "initialization_state" /storage/db/casa/webapp/hsqldb/casa.db.script
# Note the line number

# Option A: Use sed to do the replacement
sed -i 's/"initialization_state":"FAILED"/"initialization_state":"NONE"/g' \
  /storage/db/casa/webapp/hsqldb/casa.db.script

# Option B: Use vi if you prefer manual editing
# vi /storage/db/casa/webapp/hsqldb/casa.db.script
# Find: "initialization_state":"FAILED"
# Replace with: "initialization_state":"NONE"
# Save and exit (:wq)

# Verify the change was made
grep "initialization_state" /storage/db/casa/webapp/hsqldb/casa.db.script
```

```
# Should show: "initialization_state":"NONE"

# STEP 6: Clear the HSQLDB log file (forces clean state)
> /storage/db/casa/webapp/hsqldb/casa.db.log

# STEP 7: Clear admin password hash (forces regeneration)
cat > /storage/vcops/user/conf/adminuser.properties << 'EOF'
#Properties for vCOps user 'admin'
username=admin
hashed_password=
EOF

# STEP 8: Get the SHA1 thumbprint (used during initialization)
THUMBPRINT=$(openssl x509 -in /storage/vcops/user/conf/ssl/cert.pem \
  -noout -fingerprint -sha1 | sed 's/SHA1 Fingerprint=//')
echo "SHA1 Thumbprint: $THUMBPRINT"

# STEP 9: Restart services
systemctl start vmware-casa
systemctl start vmware-vcops-watchdog

# STEP 10: Wait for CASA to fully start (monitor logs)
tail -f /usr/lib/vmware-casa/casa-webapp/logs/casa.log | grep -i
'startup\|init\|error'
# Wait until you see "Started Application" or similar startup message
# Press Ctrl+C to stop tailing

# STEP 11: Trigger cluster initialization
curl -sk -X POST https://localhost/casa/cluster/init \
  -H "Content-Type: application/json"

# STEP 12: Verify cluster status
curl -sk https://localhost/casa/cluster/status
# Expected: "cluster_state": "INITIALIZED"

# STEP 13: Verify slice is online
curl -sk https://localhost/casa/sysadmin/slice/online_state
# Expected: "onlineState":"ONLINE"

# STEP 14: Wait 5-10 minutes for Gemfire cache to populate (see Issue #34)
# Then log in to the VCF Operations UI at https://192.168.1.77/
# Username: admin
# Password: Success01!0909!!
```

---

8. Quick Reference: All 35 Issues

#	Issue	Category	Severity	Fix Summary
1	PostgreSQL schema unmapped	Database	Critical	Map via `information_schema` queries
2	Credential cascade failure	Database	Critical	6-step DB repair (Issue #4)
3	API can't cancel stuck tasks	Database	Critical	`UPDATE task_metadata SET resolved = true`
4	6-step repair must be in sequence	Database	Critical	Full procedure with all SQL commands
5	PostgreSQL requires `-h 127.0.0.1`	Database	Medium	Always use `-h 127.0.0.1` flag
6	Column naming inconsistencies	Database	Medium	`state` not `status`, `resolved` boolean
7	PostgreSQL password not discoverable	Database	High	`pg_hba.conf` trust auth workaround
8	NSX needs 32GB RAM (not 16GB)	NSX	High	Set VM to 32GB RAM / 6 vCPU
9	Boot storm load >100 is normal	NSX	High	Wait 30-60 min after power-on
10	More vCPU makes it worse	NSX	Medium	Keep at 6 vCPU, increase RAM instead
11	Services need 10-15 min	NSX	Medium	Wait before API calls; use monitoring loop
12	DNS/NTP via CLI only	NSX	Medium	`set name-servers`, `set ntp-servers`
13	TEP on vmk0 (new in 9.0)	NSX	Low	Select "Use VMkernel Adapter" in Transport Profile
14	NSX cert SAN must include SDDC FQDN	Certs	High	Full OpenSSL config with all 5 SANs

15	Two trust stores need updating	Certs	High	Import to VCF trust store AND Java cacerts
16	Fleet cert generator wrong SANs	Certs	High	Full OpenSSL cert generation procedure
17	Logs cert generator wrong SANs	Certs	High	Full OpenSSL cert generation procedure
18	Shell can't handle PEM escaping	Certs	Medium	Python script to build JSON payload
19	Adapter log paths changed	VCF Ops	Medium	`/storage/log/vcops/log/adapters/`
20	JRE path changed	VCF Ops	Medium	`/usr/java/jre-vmware-17/`
21	Two separate NSX adapters	VCF Ops	Medium	Configure both VCF and Aria Admin adapters
22	Credential ROTATE broken for NSX	VCF Ops	High	Uncheck system managed, set manually
23	SSH enable via Admin UI only	VCF Ops	Medium	`https://<host>/admin/` > SSH > Enable
24	Health adapter fails silently	VCF Ops	High	Full appliance reboot required
25	Can't thin-provision to vSAN	Infra	Medium	`vmkfstools -i <src> <dst> -d thin` per disk
26	vhv.enable ghost setting	Infra	Medium	Add `vhv.enable = "FALSE"` to VMX
27	Hot vMotion fails nested	Infra	Medium	Power off VM, cold migrate, power on
28	VDT not pre-installed	Infra	Low	Download from KB 344917, SCP, unzip, run
29	Suite-API uses vRealizeOpsToken	Suite-API	High	`Authorization: vRealizeOpsToken <token>`
30	Permissions API single object	Suite-API	High	`{"roleName":"Administrator","allowAllObjects":true}`
31	Super admin empty roles	Suite-API	Low	By design — no action needed
32	domainmanager port 7200 HTTP	SDDC	Medium	Use `http://localhost:7200` not https

33	NSX credential fields uppercase	Suite-API	Medium	`USERNAME` and `PASSWORD` (not USER)
34	Gemfire cache needs 5-10 min	VCF Ops	Medium	Wait after init; use monitoring loop
35	HSQLDB reset after crash	VCF Ops	Critical	Full sed/restart/init procedure

---

9. Document Information

Field	Value
Document Title	VCF 9.0 Undocumented Issues & Discoveries Reference
Version	3.0
Author	Virtual Control LLC
Date Created	March 15, 2026
Last Updated	March 16, 2026
Total Discoveries	35
Environment	VMware Cloud Foundation 9.0.1 / Nested Lab
Issues #1-28	Discovered during initial deployment (Jan-Feb 2026)
Issues #29-35	Discovered during crash recovery (Mar 2026)

This document is part of the VCF 9.0 Lab Documentation Suite:

- **VCF9-Master-Bible.pdf** — Complete VCF 9.0 reference guide (includes all 35 issues in Appendix G.6)
- **VCF-Windows-Update-Crash-Recovery.pdf** — Crash recovery runbook (source for issues #29-35)
- **VCF-Environment-Health-Check.pdf** — Reusable environment health check
- **VCF_Troubleshooting_Handbook.pdf** — General troubleshooting procedures
- **VCF-SDDC-Manager-API-Handbook.pdf** — SDDC Manager API reference